操作系统
CHAPTER 1 概述
计算机系统
概述
电子数字计算机：能自行按照设定程序进行数据处理的电子设备。
1946年首台ENIAC。
发展：电子管计算机 -> 晶体管计算机 -> 集成电路计算机
-> 大规模与超大规模集成电路计算机
组成：软、硬件子系统
计算机硬件系统
冯诺依曼模型
总线
按功能层次分类
（按连接部件）
内部总线：连CPU内各R和运算部件
系统总线：连CPU和其他高速功能部件
数据总线
地址总线
控制总线
I/O总线：连接中低速I/O设备，通过接口与系统总线连接
通信总线：计算机间，网络
系统总线实例
处理器与寄存器
存储器
I/O控制方式轮询、中断、DMA
计算机软件系统
计算机OS
发展
手工操作阶段
作业是用户需要计算机完成的某项任务。而进程是执行的程序
的实例。一个作业至少包含一个进程
作业过程 coding -> load -> compile & link -> execute -> output
缺点
用户独占全机资源，利用率低，系统效率低
代码维护难（穿孔卡片、纸带）
输入、执行、输出均联机进行，耗时长
半自动化控制方法
联机I/O技术
操作员操作，由管理程序将作业输入到磁带上
管理程序自动将磁带上的1st作业装入内存并执行
执行完一个作业，管理程序继续处理下一个
脱机I/O技术
一主机，一辅机。辅机仅用于I/O，不与主机连接。
作业通过辅机输到磁带上（脱机输入）
主机从磁带上读作业到内存上并执行，完成后再输出到磁带上
（脱机输出）
辅机打印输出
操作系统及其分类
操作系统是配置在计算机硬件上的第一层系统软件，负责管理
和控制计算机的硬件与软件资源，提供用户与计算机之间的接
口，并为其他软件的运行提供支持环境。
组成
分类从操作控制方式角度
多道批处理操作系统，脱机控制方式
批处理（Batch）操作系统：脱机、成批处理、单/多程序运行
Batch多指非交互式计算，如.sh, .bat等
分时操作系统，交互式控制方式
允许多个联机用户使用一台计算机系统OS
一台主机连接多个终端，主机CPU轮流给每个中断一个时间片
特性：同时性、独立性、及时性、交互性
实时操作系统
前台作业：交互型
后台作业：批处理型
典型
分时OS为用户提供通用交互型开发运行环境，实时OS提供特
殊用途专用系统
深入观察OS
资源管理角度
资源：硬件资源+信息资源（程序与数据）
驱动程序：最底层、直接控制和监视各类硬件（或文件）资
源。隐藏底层硬件细节，提供抽象、通用接口。
资源共享方式：独占&并发
资源分配策略
静态分配：进程运行前一次分配全部所需资源分配简单，但是利用率低
动态分配：使用资源前临时申请利用率高，实现复杂，可能发生死锁
资源抢占：需要资源时，强制收回分配给优先级更高或等待时
间长的进程避免死锁，但被抢占进程状态不稳定
资源管理主要技术
资源有限
复用（解决资源利用不充分）
空分复用：空间上分割成更小单元使用。如内存划分、硬盘划分
时分复用
虚拟（解决资源不足）把你一个物理资源转变为逻辑上的多个对应物
实现复杂抽象
进程抽象
针对进入主存的程序在处理器上的操作状态集
如表面的多进程并发执行、wait（）等系统调用背后的处理器
操作
虚存抽象
针对主存和磁盘对换文件
意义
1. 每个进程的虚拟地址空间从零开始，便于编程和管理
2. 隔离不同进程
3. 通过页表映射和换页，可以扩展内存，部分数据放在磁盘
上，需要时再调入（时间局部性+空间局部性）
文件抽象针对设备：存储设备与传输、控制等设备
程序控制角度
多道程序设计（multiprogramming）：允许多个程序同时进
入主存，竞争处理器资源。提升使用效率。（CPU速度与I/O
速度不匹配）
优点：提高 CPU 利用率和系统整体吞吐量；
代价：单个作业的周转时间（turnaround time）可能变长。
管理开销：页表查找、缺页中断处理、换页操作都需额外时间
等待时间：分时共享，不能独占资源
缺页延迟：如果数据被换出，调入磁盘很慢
实现要点：如何使用资源、如何复用CPU（调度）、如何使
CPI与I/O充分并行、如何让程序让出CPU（中断）
考虑问题：处理器管理与分配、存储管理与存储保护、其他资
源管理与调度
操作控制计算机角度
OS的操作接口（系统程序）提供工作流程，为用户提供操控
计算机的所有服务
两类作业级接口
脱机作业控制方式
定义：I/O 操作不再由主机直接控制，而是交给外围机（I/O
处理机），主机只与磁带/磁盘等高速设备打交道。
批处理系统
如Shell程序、bat程序
联机作业控制方式
定义：输入/输出设备直接和主机相连，由 主机直接控制 I/O
操作。
注：“机”指的是输入/输出机（外围机），也就是专门用于
I/O 的计算设备。
命令解释程序
操作系统里负责接受和解释用户输入命令，并把它转交给系统
去执行的程序。（批作业、交互型系统）如CMD
处理过程
提示符显示 → 读取输入 → 解析/扩展 → 判断命令类型 → 执
行 → 返回状态 → 循环
前台命令：用户输入后，命令立即运行，并且占用终端（控制
台），用户必须等待它完成才能继续输入其他命令。
后台命令：用户输入后，命令在后台运行，不会阻塞终端，用
户可以继续输入其他命令。
人机交互角度
OS的人机交互（Human-Computer interface， HCI）是用
来改善人机界面，为用户使用计算机提供方便环境。
人机交互设备
传统的终端设备：通过键盘、鼠标&显示屏操作计算机
新型的模式识别设备：语音、动作等识别设备
用户界面
命令行
图形
触摸屏
语音
发展：行命令控制 -> 全屏幕控制命令 -> 以人为中心（鼠
标、菜单&窗口控制）-> WIMP （Windows, Icons, Menu,
Pointing Devices）
程序接口角度系统调用
为程序设计者提供的易于使用的计算机访问接口
用户程序访问OS服务的唯一途径
作用：内核基于权限和规则对资源访问进行裁决，保证系统安
全性；对资源进行抽象，提供统一接口。
实现
实现机制：系统陷阱（系统异常处理机制）
由于系统调用而硬气处理器中断的机器指令为陷入指令（Trap）
每个系统调用都事先规定了编号，在约定寄存器中规定传递参数
实现要点
1. 编写系统调用的处理程序
2. 设计系统调用入口地址表：入口地址指向系统调用的处理程
序，包含自带参数的个数
3. 开辟处理器现场保护区
系统调用与函数调用
系统结构角度
OS结构
单体式
按功能模块为基本单位
每个功能具有一定独立功能，若干模块协作完成某功能
明确各模块间接口关系，模块间自由调用
各模块分别进行设计、编码、调试，再整合
定义：整个操作系统作为一个大整体，所有功能模块都运行
在 内核态，共享同一个地址空间。
层次式
定义:把 OS 按 功能层次 划分，每一层只依赖下层、服务上
层。
严格分层，层与层之间接口明确。
类似洋葱结构，从硬件 → 驱动 → 内存管理 → 文件系统 →
用户接口。
虚拟机
定义：在底层硬件之上加一个 虚拟机管理层，把物理机虚拟
成多个独立的“虚拟机”，每个虚拟机都像一个独立的计算
机，可运行完整 OS。
提供 完全隔离 的运行环境。
上层操作系统在“虚拟硬件”上运行。
微内核
定义：把 OS 内核功能 尽量缩小，只保留最基本的（进程管
理、内存管理、通信 IPC）。其余功能（文件系统、驱动、网
络等）都作为 用户态服务 运行。
内核小而精简。
内核与服务之间通过 消息传递（IPC） 通信。
操作系统内核
CHAPTER 2 处理器管理
处理器(基本硬件知识)
处理器及其部件
寄存器
用户程序可见寄存器
使用机器语言编程时合理使用此类寄存器课减少贮存访问次
数, 提升效率
控制与状态寄存器
程序状态字(PSW) 描述处理器现场的程序执行的动态行为
指令与处理器模式
机器指令(指令)
执行: 取指(PC-M-IR)->解码(ID)->执行
采用指令流水线执行
程序分为:
- OS内核程序: 有较高特权, 可使用all机器指令
- 普通应用程序: 权限较低, 不能执行有关资源管理的指令
特权指令: 只有内核态可以用
非特权: 用户态也可以用
处理器\主存\设备等资源管理程序在内核态运行, 保障安全性
应用程序逻辑和数据处理在用户态执行
设置有硬件标志位: 处理器状态位, 来区分状态
处理器状态分4种
0级: 内核态
1级: 系统调用
2级: 共享库
3级: 应用程序
用户态->内核态转换条件: 中断
异步中断事件(如I/O完成), 运行程序被中断, 转向中断处理程
序
实现对外部设备的及时响应，提高 CPU 与 I/O 并行工作效
率。
执行系统调用, 称为自愿性中断
程序自身主动请求操作系统服务（比如：读文件、创建进
程）。
执行指令出现问题产生异步异常事件(如用户态执行特权指令),
程序被打断, 称为程序性中断事件
保证系统安全和稳定，OS会处理错误或提供机制（比如缺页
异常时换入页面）。
用户栈: 存放用户态程序运行时数据, each进程独立用户栈
内核栈: 进程因 系统调用 or 中断/异常 进入内核态时, 系统切换到该进程的内核栈. 同样each独立
中断
BASIS
定义: 打断正在执行的程序，让CPU转去处理更紧急/更重要的事件
广义中断：
狭义中断：源于处理器外部的异步中断事件
异常：运行中程序出现异常，由CPU控制器引向异常处理电路
的同步中断事件
系统异常（系统调用）：异常的一种，但被引向正常处理电路
分类
内中断（异常）
故障指令执行问题，如缺页、除零
自陷事先安排的异常，如系统调用
终止硬件出错，如控制器、存储器校验出错硬件中断
软件中断
外中断（中断）
可屏蔽中断 INTR
不可屏蔽中断 NMI
中断装置： 发现中断源并产生中断的硬件
中断源多样 -> 多种中断装置
分类
处理器外的中断：由中断控制器发现和响应
中断控制器: CPU中, 包括中断控制逻辑线路和中断寄存器
过程: 外部设备发出中断请求 -> 中断寄存器设置已发生中断 -> 指令处
理结束前, 会检查中断寄存器 -> 有不被屏蔽的中断则引出OS的中断处
理程序
在当前指令执行完毕后，在CPU准备取下
一条指令之前，会进行中断检查。
处理器内的异常：由指令的控制逻辑和实现线路发现和响应，
相应机制称为陷阱
陷阱 (Trap) 是一个广义术语。它指的是任何由当前正在执行
的指令所引起的、同步的、从用户态到内核态的控制权转移。
它就像一个预设的“陷阱”，一旦CPU执行的指令踩上去，就
会立即把控制权交给操作系统。
CPU捕获异常(陷阱的一类)后，会立即跳转到OS预设的异常处
理程序。分析异常类型，然后决定如何处理：终止程序，or
修复问题后继续执行。
页面异常(虚拟内存页面不在物理内存中)发生, OS的页面异常
处理程序被激活, 修复页表. 处理后, 回到一场指令重新执行.
请求OS服务的系统异常：处理器执行陷入指令时直接触发，
相应机制称为系统陷阱
当用户程序需要执行一些高权限操作时（如读文件、创建进程），它
不能直接操作硬件，必须请求OS帮忙。
通过执行陷入指令（如 SYSCALL）来主动触发一个陷阱。
系统调用完成后，控制权会返回到陷入指令的下一条指令继续执行。
中断源：引起中断的事件
处理器硬件故障中断事件
程序性中断事件：执行机器指令引起，如除以0、溢出、非法
指令、终止进程指令
自愿性中断事件：系统调用，处理器执行陷入指令请求OS服
务引起中断
I/O中断事件: 用于报告IO状态, 如 IO 完成\出错\异常
外部中断事件: 外围设备发出, 时钟中断\键盘信号中断
中断系统
硬件子系统(中断响应)
软件子系统(中断处理)
中断响应过程
中断响应过程 是一个由硬件自动触发，并最终将CPU控制权
移交给操作系统软件的标准化流程。其核心是在保证当前指令
原子性的前提下，安全地暂停当前程序，并精确地跳转到预设
的内核代码来处理紧急事件。
过程
请求阶段：外部或内部事件源向中断控制器发出中断请求。
仲裁阶段：CPU 在每条指令执行末尾查询中断控制器。控制
器负责屏蔽无效请求，并从所有有效请求中根据优先级选择最
高的一个。
响应阶段：CPU 决定响应中断，暂停当前程序的执行。
保存现场：CPU 硬件自动将当前程序的程序计数器(PC)和程序
状态字(PSW)等关键上下文压入内核栈。
转向处理：CPU 通过中断向量查询中断描述符表(IDT)，找到
并跳转到操作系统预设的中断处理程序的入口地址，开始执行
内核代码。
多中断响应 & 处理
中断屏蔽
中断优先级
中断嵌套处理
中断的响应与处理
进程管理
BASIS
进程是进程实体的运行过程，是系统进行资源分配和调度的一
个独立单位
特征
动态性
并发性
独立性
异步性进程是动态的，程序是静态的
可再入程序：可被多个程序同时调用的代码段故进程可用来共享资源
可再用程序：被调用过程中会修改自身，不可多程序同时调用
进程控制
状态
进程实现的队列模型
创建: 进程表加一项 -> 申请PCB并init -> 生成标识,建立映像,
分配资源 -> 移入就绪队列
撤销: 移出队列 -> 归还资源(先撤销all子孙进程),撤销标识, 回
收PCB -> 移除表项
阻塞: 保存现场信息, 修改PCB, 移入等待队列, 调度其他进程执
行
唤醒: 移出队列, 修改PCB, 移入就绪队列
挂起：在资源不足以满足进程要求时，剥夺内存及其资源，调
入磁盘对换区，不参与进程调度，待适当时候回来（区别于阻
塞态（等待态）占有一定资源）
切换: 保存被中断进程上下文,更新PCB并加入相应队列->许纳
泽另一个进程,更新PCB,更关心内存管理的数据结构,恢复上下
文
模式切换
数据描述
PCB 进程控制块
存放标识信息，现场信息，控制信息
程序使用内存
正文段：代码+赋值
堆段：动态分配
栈段：临时变量
进程映像：某一时刻进程的内容及其执行状态的集合
进程上下文: 进程物理实体和运行环境
OS执行模式
OS服务例程：OS用于提供特定服务的一组函数 、程序。用户
可以通过系统调用请求这些服务
两种模式
服务例程嵌入应用进程中运行
服务例程作为独立进程运行模块化，接口简洁
功能独立，便于OS实现、扩展
适用多处理器环境，一些系统服务可以放到专门处理器上
多线程技术
BASIS
进程目的：更好地并发执行多道程序，提高资源利用率和系统吞吐量
线程目的：减小并发程序时空开销，提高并发性能
组织方式
调度员、工作者模式
调度者线程检查后选择一个工作者线程唤醒
组模式
所有线程平等
流水线模式
并发多线程程序优点：快速线程切换、易于通信、减少管理开
销、并行程度高
实现方式
内核级线程 KLT
线程的创建、撤销、切换全由内核完成
优点：并发性强、支持多处理器、内核自身多线程
缺点：模式切换开销大
用户级线程 ULT
通过线程库执行
优点：效率高（不用模式切换）、灵活调度、可移植性强
缺点：阻塞会阻塞整个进程、无法利用多核
混合策略
结合 ULT 和 KLT 的优点。一个应用中的多个用户级线程映射
到少量内核级线程上
处理器调度
层次
作业调度（Job Scheduling）四状态
输入状态
作业从外部介质提交到系统时，先进入输入井。
正在等待系统把它调入内存。
后备状态
暂时存放在外存的“后备作业队列”中。
此时作业 还不是进程，只是等待进入内存的“作业实体”。
执行状态
这时它已经转化为一个或多个进程，由进程调度分配 CPU 来
运行。
完成状态释放资源，输出信息
三级调度
高级调度：作业能否进入内存（从外存到内存）。保证系统的吞吐量与响应时间，维持系统负载的均衡。
中级调度：进程能否留在内存（挂起与恢复）。缓解内存紧张，提高系统资源利用率。
低级调度：哪个进程获得CPU执行（在内存的就绪进程中挑
选）。
保证CPU高效利用，满足不同调度算法（FCFS、RR、优先级
等）
“外存作业 → 高级调度 → 内存进程 → 中级调度 → 就绪队列 → 低级调度 → CPU运行
算法
考虑原则
资源利用率 CPU利用率=有效时间 / 总时间
吞吐率: 单位时间CPU处理的作业数
公平性: 确保all进程分配合理资源, 不会饿死
响应时间: 从请求到响应的时间
作业周转时间: (批处理)从作业提交到完成的时间
调度算法
1. FCFS(First Come First Served)
效率低，优待长作业，不利短作业。适用CPU繁忙型作业，不
适用I/O繁忙型
e.g.
2. SJF(Shortest Job First)
效率低, 会出现饥饿. 需要估计作业所需时间
平均等待时间、平均周转时间最优
e.g.
T = (3 + 12 + 40) / 3 = 18.3
W = (3/3 + 12/9 + 40/28)/3 = 3.76
Time estimation:
if 允许抢占, 会使长进程产生饥饿
忽视作业等待时间
3. SRTF(Shortest Remaining Time First)
最短剩余时间优先
e.g.
4. HRRF (Highest Response Ratio First)
响应比=1+已等待时间 / 估计运行时间
不会饥饿
e.g.
性能优于SJF, 差于SRTF
5. PS (Priority Scheduling)
选取优先级最高的作业. 可以用户也可以系统指定优先级, 分静
态和动态(随时间而改变)优先级.
重要程序\交互程序优先级大
动态优先数法
两种原则:
1. 短进程优先(执行时长)
2. 久进程优先(等待时长)
6. RR (Round Ribbon)
(环形)就绪队列中轮流分配一个时间片
改进轮转法: 不同进程不同时间片数
进程切换上开销较大(取决于时间片大小)
7. MLFQ (Multi-Level Feedback Queue)(分级调度算法)
优先级分层：高优先级队列时间片短，低优先级队列时间片长。(第1级 优先级最高)
反馈机制：新进程先进入高优先级队列；若在时间片内未完成，则被降到更低优先级队列。
抢占式调度：高优先级队列中的进程会抢占低优先级队列中的进程。
提升机制（可选）：长期处于低优先级的进程会被提升，以避免饥饿。
all 进程首先进入第1级队列, 分配1时间片 ;
之后进入第2级队列, 分配2时间片;
......
第1级队列优先级最高, 最紧急, 分配1时间片, 保障快速响应
性能较好, 利于分时交互作业(响应快), 利于长批处理作业(不会饥饿)
8. 彩票调度算法
基于概率调度, 每个进程持有一定数量的彩票, 根据数量配置平
衡资源使用
CHAPTER 3 存储管理
内存管理主要功能
内存空间的分配与回收
地址转换
内存空间的扩充
内存共享
存储保护
存储器
上部, 在掉电后信息丢失
磁盘缓存, 即虚拟内存
CPU 只能直接从主存中读取指令和数据来执行。
so 可执行程序必须存在主存中 (但由于虚拟内存技术的应用, 每次只需将
需要的部分程序放在内存中, 暂时不用的先放磁盘里)
编译
源程序 --编译/汇编程序--> 多个目标模块如: .c文件 --> .o文件
编译程序记录模块内引用的发生位置
目标模块附有供引用使用的内部符号表和外部符号表
符号表: 每个目标模块一个, 记录模块的内部符号和外部符号
内部符号: 模块中定义的符号, 指函数名和变量名, 其中局部的
不导出, 对其他模块不可见
外部符号: 模块引用的其他模块的符号
每个目标代码从0号单元开始编址
链接
作用: 将多个目标模块链接成一个完整的可重定向的程序
静态：运行前链接成一个完整的装入模块
装入时动态：边装入边链接
运行时动态：执行需要时，再链接
顺次连接各模块地址，构成统一的从0号单元开始的逻辑地址
空间
解析符号表, 将对符号名的引用转换为数值, 将程序入口点和数
据引用点转换为数值地址
可重定位目标程序 (装载代码模块): 存放在磁盘中，不知道将放在主
存的什么位置 ，so 程序地址总是从0开始，称为逻辑地址
装入
分配主存, 将逻辑地址绑定物理地址(称为 地址转换 or 重定向)
重定向
绝对装入：编译时直接转为绝对地址（only单道程序环境）
静态: 一次性将all 逻辑地址转变为物理地址
动态: 执行指令需要时再转换. 须借助硬件的地址转换机构
空间存储
连续
固定分区
内存空间静态分割成若干分区, each分区 one 作业(程序一次
性装入)
主存分配表
碎片
内碎片: 分区内部剩余空间
外碎片: 分区间空间<程序需要 而被丢弃
作业载入
每个分区有单独的等待队列, 装入时选择能装入的即可 or 选择
能装入的最大作业
地址保护
设置上/下限寄存器
下限寄存器(基址寄存器): 存储物理起始地址
上限寄存器: 物理中止地址
缺点: 内部碎片严重; 进程大小受限; 并发进程数受限(固定数量分区)
可变分区
按需分配, 分区与请求大小一致. 找到合适空闲分区再分割所需大小
内存分配数据结构
空闲分区表: 列表存储分区序号\起始地址\分区大小等数据项
空闲分区链: 链表存储
空闲分区分配算法
1. 首次适应: 空闲分区链按地址增序链接, 从链首顺序查找第一
个合适的分区进行分割
2. 循环首次适应: 从上次查找的分区下一个开始查找
3. 最佳适应: 分区按大小顺序链接, 找到满足要求的最小分区
4. 最坏适应: select最大分区分割给作业使用(使得分割剩余空
间不至于太小) 利用率差, 大作业难装入
5. 快速适应: 为常用长度的分区, 设置单独空闲分区链表
地址转换: 基址寄存器+限长寄存器
缺点: 外部碎片严重; 管理复杂;
伙伴算法
二分 , 每块大小2^i
数据结构: 双向链表记录空闲块
e.g.
地址
离散
分页式
逻辑连续, 物理离散
减少内部碎片
e.g. 20KB, 原来存入32KB, 浪费12KB
现在可以存入5个4KB的页面, 0浪费
避免分区合并(原来分区大小固定, 若过小, 则需合并几个分区)
进程运行时，每个阶段只需要部分代码块，所以通过离散的方
式，可以将暂时不需要的代码移出内存
逻辑地址->物理地址
页面 -> 页框(内存块\ 物理页), 二者大小一致
逻辑地址位数 比物理地址大
利用局部性原理, 离散使得可以每次只将所需内容放入内存, 不
需要的内容等需要了再从外存中调入内存.
计算题
逻辑地址A, 页面大小L, 页号P, 页内地址d
e.g.
P = 1023 / 1024 = 0 => 2 (page 0 映射到 page 2) 注意这里的逻辑地址为十进制
d = 1023 % 1024 = 1023
A' = P*L + d = 2 * 1024 + 1023 = 3071
页表
页表放内存, CPU每存取一个数据, 两次访存
1. 访页表取物理块号
2. 根据物理地址存取数据这使得效率下降一半, 所以我们引入了快表TLB
多级页表
现代系统支持的逻辑地址空间非常大, 采用多级页表降低空间
占用
核心约束：在多级分页系统中，每一级页表必须能够刚好放入
一个物理页框中（或者不超出一个页框），以便离散存储。
物理机制
逻辑地址结构
页目录(选择页表)\页表页(选择页面)\位移(定位页框内地址)
外层页表也叫一级页表
CPU页表基址寄存器存储的是一级页表的起始物理地址
反置页表 IPT
整个OS只有一个反置页表
“反置页表”是一种用“CPU时间”（查找）来换取“内存空间”（存储）的策略。它牺牲了
“TLB未命中”时的查找速度（变得更慢），来换取（在64位系统中）极其宝贵的内存空间。
结构
传统页表（正向）： 以“逻辑页号”为索引。 页表[逻辑页号] -> 物理帧号。
反置页表（反向）： 以“物理帧号”为索引。反置页表[物理帧号] -> <进程ID, 逻辑页号>。
机制
必须搜索全局IPT，逐行查找内容 <PID 5, 逻辑页 10>
解决方案
哈希表: 哈希函数 hash(PID, 逻辑页号) 来“猜”一个索引 i
比线性搜索快, 但是仍比传统直接计算慢
快表（相联存储器）
作为“页表”的高速硬件缓存
利用局部性原理: 时间+空间
可避免访问内存页表的低效
位图
通过位图标注每个页面是空的还是满的, 一位表示一页
假定4GB内存, 有2^32字节
假定每页1KB, 有2^22页, 一页用一位来表示
即2^22 bits = 2^19 bytes = 512 KB 利用512KB即可描述内存4GB的使用情况
页面共享
多进程可复用相同程序段页面, 比如各种代码库
正常情况：共享页/段的逻辑页号/段号可以不同
特殊情况: 共享代码在编译时，被假定在一个绝对的“逻辑地址”上（例
如，CALL 0x1000）。
为了让这个模块被多个进程共享，这所有的进程都必须在它们各自的逻辑
地址空间中的同一个“统一”地址（0x1000）上映射这个模块。
在共享模块增多时，会引发逻辑地址冲突，管理起来“非常困难”。
页面保护: 在页表中增加一些标志位，用来指出该页的
信息可读\写\只读\只可执行\不可访问等
动态链接
动态链接是指，运行时把共享库中共享函数的目标代码库加载
到程序的主存区的过程
是一种程序链接方式，它将“链接”这个过程——即“解析和绑定”
外部函数与库的过程——从“编译时”推迟到了“运行时”。
分段式
二维逻辑地址
将程序若干逻辑段拆分用户可见
OS为每个作业建立段表, 记录每个段的段号\内存地址\段长
段表控制寄存器: 分段存储器中存储
当前作业段表的起始地址和长度
图中缺少: 段号的越界检查
比较: 比较限长域实际段内地址, 判断是否地址越界
段号决定段表行号
段的共享与保护
优势
“段”是一个逻辑单位。程序员（和编译器）在编译时就已经将程序划分
为“代码段”、“数据段”、“堆栈段”。
操作系统可以在“段”的级别上设置权限：
- Code Segment：设置为 Read-Only / Execute-Only
- Data Segment：设置为 Read / Write
这种保护完全贴合程序的逻辑结构，比“页级”保护更安全、更合理。
共享变得非常显式和自然。
OS 只需将 "editor" 代码加载到物理内存一次。
- 进程1 的段表："Editor" 段 -> 指向物理基址 80。
- 进程2 的段表："Editor" 段 -> 同样指向物理基址 80。
这种“按逻辑名称”(Editor 段) 来共享的方式，远比“按一堆
页号”来共享要简单和清晰。
对比分段与分页
分页是物理一刀切，分段是按照程序员逻辑分割
段页式
结合分页(提高内存利用率) & 分段(满足用户需求)
段页式: 将段分成若干页
虚拟存储管理
传统存储管理方法：一次性（作业一次装入内存）& 驻留性
（作业驻留内存直至运行结束 ）
虚拟存储：程序装入时，仅装入所需的少数页面，通过请求调页
和页面置换，为用户提供一个比实际内存容量大得多的存储器
局部性原理：时间 + 空间分析程序：顺序执行为主，包含循环，连续过程调用少等
虚存大小 受限于地址结构 & 内外存容量和
D: 在具有层次结构存储器的计算机系统中，采用自动实现部
分装入和部分对换功能，为用户提供一个比物理内存容量大得
多的，可寻址的一种“内存储器”。
实现方式
请求分页式
硬件支撑
内存管理单元 MMU：将虚拟地址翻译成物理地址
管理&访问页表
分解逻辑地址为页号&业内地址
管理快表
发出缺页中断&越界中断
管理特征位（状态位、修改位etc）
过程
页表、缺页中断机制 Page Default
基本原理
动态页面装载（装入部分页面）
页表结构驻留标志：用来标记这一页是否在物理内存中
辅存地址：当发生缺页中断时，操作系统读取这个地址，把页
面从硬盘调入内存
交换区：硬盘上预先划分出来的一块特殊空间作为后备内存
换入 (Page-In)：当发生缺页中断时，OS 从交换区把页面读
入内存。
换出 (Page-Out)：当内存已满, 须调入新页面时, OS选择一个
页面调出至交换区
页面装入清除策略
页装入策略：何时将一个页面装入内存
按需调入, 一次调一页, 节省空间;
but 开销大
一次调入多页, save time
but if 调入页面均未被使用, 则效率低, 故引入预测机制
页清除策略：合适将修改过的页面写回外存
请页式清除: 页面被替换出内存时, 才检查是否被修改过, 如果
是,则写回辅存
预约式清除: 后台线程主动检查, 在替换前就写回了
页面分配策略
给进程的分配数量：尽量减少缺页中断次数
固定分配：进程在生命周期中固定物理块数，不同进程可不同
数量（平均、按比例、优先权分配）
可变分配：不断评价进程的缺页率，改变进程的页框数
替换粒度：置换时，局限于本进程 or 整个内存空间
每个进程的页面数：过多（并发进程减少），过少（缺页中断增加）
页面淘汰算法：选择被淘汰页替换掉
抖动（颠簸）现象：刚淘汰的页面又调入，频繁发生
全局页面替换策略
FIFO
实现简单、适合线性特征程序
but 缺页中断率高
伴有Belady异常：增加主存块，缺页会更多
最佳页面算法 OPT
淘汰距今最久再访问的页（最晚需要的）
实际不可预知，故无可行性。用于算法优劣衡量
最近最久未使用 LRU（Least Recently Used）
根据时间局部性原理，淘汰最久没用的
实现
引用位法：each页一个标志位R，if 访问该页，R置1，隔一段
时间全部清0（period选择很关键）。淘汰R=0中的一页
计数器法：each页一个计数器，访问一次+1。淘汰值最小
者。淘汰后全清0。
计时法：each页一个计时器，计入最近一次访问绝对时间。淘
汰最老页。定期清0。
寄存器法：each页一个多位寄存器，if 被访问，最左位置1，每隔
时间t， R右移一位。淘汰R值最小者。（老化算法，性能较好）
双向链表（最常用）：每次引用，动态移动
第二次机会（时钟页面替换算法，转圈像钟）：FIFO改进，
add引用位R，if R=0，则淘汰，else 移到末尾并置0. 如果被
引用置1.（利用单向循环链表）
Clock（改进时钟算法）
联合引用位与修改位（r，m），r表最近是否被引用，m表是
否 被修改
实现
1. 扫描循环队列，淘汰（0,0）
2. if 1失败，找（0,1），扫描过程中所有r置0 首先关注是否访问过
3. if 2失败，重复1 & 2
局部页面替换策略
局部最佳 MIN算法：当发生缺页中断时，替换掉在未来最长
时间内不会被访问的那个页面。 but 因无法预知而实用性差
工作集置换算法
工作集：一个进程在当前时间 t 的工作集WS(t, Δ) 是指在过去 Δ 时间窗口内它
所访问过的页面的集合。Δ 称为“工作集窗口大小”。
在发生缺页中断时，选择一个不在当前工作集中的页面进行替换。
but 算法实现困难且开销大. OS 需要知道哪些页面在工作集
中, 且须为每一个页面维护一个最后访问时间戳.
模拟工作集替换算法
1. 页面引用时, 引用位置1
2. 周期性检查引用位: if 为1, 清零并设置当前时间; else 计算
自上次检查未被引用 至今的时间
3. if 时间超过阈值, 则移除
缺页频率替换算法
请求分段式
请求段页式
CHAPTER 4 设备管理
I/O 硬件原理
设备管理的目的：克服设备与CPU速度不匹配
I/O设备分类
按I/O特性：输入、输出型，存储型（顺序存取（如磁带）、直接存取（如磁盘））
按信息交换单位：字符（输入、输出型）、块设备（存储型）
控制方式
轮询
use 查询指令测试设备控制器的忙闲标志位，以确定主存&设
备是否能交换数据
轮询的关键在"就绪"部分, 不断查询, 直到就绪
轮询指令
1. 查询指令：查询设备是否就绪
2. 读写指令：当设备就绪时，执行数据交换
3. 转移指令：当设备未就绪时，执行转移指令转向查询指令继续查询
缺点：
等待时浪费CPU：在I/O设备准备数据的过程中，CPU必须持续地、主动地去查询设备状态
传送时占用CPU：当I/O设备终于准备就绪后，CPU必须暂停原程序，亲自参与数据的传送工作。
中断
硬件要求：
①CPU与设备控制器及设备之间存在 中断请求线
②设备控制器的状态寄存器有相应的 中断允许位
过程
1. 进程发出I/O信号(系统调用), 控制信息->设备控制器寄存
器. (进程继续执行其他任务 or 放弃CPU等待I/O完成)
2. 设备控制器检查寄存器内容, 按照指令执行. 完成后, 通过中
断请求线发出I/O中断信号
3. CPU收到并响应, 转向设备的中断处理程序执行
4. 中断处理程序数据读取后, 将缓存寄存器的内容写入主存
5. 进程调度程序将得到数据的进程在适当时刻恢复
CPU启动I/O命令->控制器管理设备->返回程序->中断处理程
序->返回断点
优: 在设备准备数据期间, CPU仍可继续执行程序, 提升利用率
缺: 每传输一个字, CPU就被打断一次. 保护现场\恢复现场\执
行搬运指令, 浪费时间
DMA
所需逻辑部件
MAR: DMA每交换一次数据, 地址寄存器+1
字计数器(DC): 记录传输总字数
数据缓冲寄存器: 暂存每次传送的数据
设备地址寄存器
中断机制和控制逻辑
控制器
过程
CPU告知DMA控制器 读写相关信息 -> DMA挪用总线, 搬运
数据->搬完整块, 发中断
通道
流程
1. CPU遇I/O, 启动指定通道上的设备
2. CPU与通道并行. 待通道发出结束中断.
类型
设备控制器
I/O设备包括机械部件(设备本身)和电子部件(设备控制器 or 适配器)
PC的CPU和控制器间为单总线, 便于直接控制
大型主机采用多总线和通道, 提高CPU和输入输出的并行性
组成
接口
功能
1. 接受&识别 CPU或通道的命令
2. 数据交换
3. 记录设备和自身状态(for CPU)
4. 识别设备地址
I/O 软件原理
设计原则: 高效\通用
组织层次
用户调用 -> 查 Cache -> 没命中则阻塞 -> 驱动发命令 -> 硬
件干活 -> 硬件发中断 -> 中断唤醒进程
中断处理程序
运行态 --(请求I/O)--> 阻塞态 --(I/O完成中断)--> 就绪态
不仅处理“成功”，由于硬件的不稳定性，它必须包含复杂的
错误处理逻辑（Retry、Abort、Report）。
设备驱动程序
将用户提供的逻辑IO请求转化为物理IO操作
功能: 设备初始化\执行程序\执行中断处理程序
处理过程: 同步 I/O (阻塞) vs 异步/快速 I/O (无须阻塞)
大多设备很慢, 需要阻塞读写
有些设备极快, 不阻塞
程序后程: 被中断唤醒后, 验货 -> 上交 -> 接下一单
独立于设备的I/O软件
统一命名：设备即文件 (基于 inode)，统一用 open/read 接口。
统一尺寸：屏蔽物理扇区差异，向上一律提供统一逻辑块。
缓冲技术：在内核开辟缓冲区，解决速度不匹配问题。
为解决以下矛盾：
- 缓和速度差异
- 协调数据大小不一致
- 提高并行性
单缓冲
设备输入T和 CPU 处理上一个数据C可以并行
M结束后，T才可以再往缓冲区输入
M结束后，C才可以处理
so，C和T并行，且时长为二者 max
处理时间
无缓冲时：T + C
单缓冲时：max[C, T] + M
双缓冲
T1，T2交替输入，如果T很长，则平均时间为T
否则，就是M+C更长
M<<T, 所以通常近似比较
多缓冲（循环缓冲：环形链表）
分为三种状态：空缓冲区（等着装数据）、满缓冲区（装着数
据等着取）、工作缓冲区
缓冲区高速缓存
为了 “减少磁盘 I/O 次数”
内核中划出一块内存（高速缓存），根据局部性原理保存磁盘
数据块
设备分配：负责设备的静态、动态、虚拟(SPOOLing) 分配。
静态分配：作业开始前就独占设备（效率低，简单）。
动态分配：运行时按需申请（效率高，可能死锁）。
虚拟分配：SPOOLing 技术（把独占设备模拟成共享设备）。
错误报告：驱动搞不定的错误，由它向上层汇报。
用户空间的I/O软件
库函数：封装系统调用成简单函数
SPOOLing技术 假脱机
Simultaneous Peripheral Operation On-Line（外部设备联机并行操作）
核心目的：解决 CPU 的高速性与 I/O 设备的低速性之间的矛盾
在联机情况下，用软件模拟脱机输入输出。
输入井/输出井：在磁盘上开辟两块区域，一个存输入数据，
一个存输出数据。
优点：
1. 提高IO速度：将磁盘作为缓冲，将对设备的操作转变为对磁盘的操作
2. 将独占设备改造成共享设备：可同时接受多个进程的 IO需求，存起来排队处理
3. 实现虚拟设备功能：每个进程都感觉独占设备
驱动调度技术
目的
磁盘时共享设备, 同时被多个进程请求. 而磁盘读写需要机械运
动, 需要考虑请求顺序, 来减少寻道时间&旋转延迟, 提高系统
吞吐量.
存储设备的物理结构
顺序存取(磁带) 严格依赖物理位置。读后方信息必须先读前方信息。
优点是容量大、可靠、可保存；缺点是慢。
直接存取(磁盘)
物理地址 = 柱面号 + 磁头号+ 扇区号
提高磁盘 I/O 速度的五种方法
循环排序与优化分布(减少旋转延迟)
案例: 处理同一磁道上多个记录的读取请求
按IO请求次序
例题: 请求顺序4321, 排列顺序->
注意: 是磁盘旋转, 不是磁头
耗时: 1/2(平均定位时长)+1/4(读取记录4)+3/4*3(读取另外三
个记录)=3周
按数据存储方向
读取顺序为1234(数据存储时的顺序)
耗时: 1/2+1/4*4=1.5周
根据当前磁针位置, 调整记录顺序
已知当前位置为记录3, 故读取顺序为4123
耗时: 1周
优化分布
案例: 读出每个记录后要花时间处理, 处理完才能继续读
调整分布, 读完A,处理一段时间, 正好继续读B
交替地址 (多重副本): 相同数据, 有几个交替地址, 读写更快
搜查定位 (减少寻道时间) 移臂调度策略
FCFS 寻道时间周期长，性能较差
最短寻道时间优先 SSTF 产生饥饿
电梯调度 SACN
循环扫描 C-SCAN
磁头单向移动
分布扫描
将请求队列分成长度为 N 的子队列。子队列内部用 SCAN，
子队列之间用 FCFS。
RAID 独立磁盘余阵列 (并行读写)
用一组容量小、独立的、可并行工作的磁盘驱动器组成阵列来
代替单一的大容量磁盘，再增加冗余技术，数据分布存储，单
个 IO 请求能并行地从多个磁盘驱动器同时存取数据
类别
RAID 0 (无冗余)：
原理：条带化 (Striping)。数据被切块，分散存到所有盘。
特点：速度最快（并行），无容错（坏一个全死）。
RAID 1 (镜像)：
原理：镜像 (Mirroring)。每个盘都有一个完全一样的备份
盘。
特点：安全性最高，读取快（可并行从两个盘读），利用率最
低 (50%)。
RAID 2 (海明码)：
原理：按位 (Bit) 分割数据。利用海明码 (Hamming Code)
进行纠错，校验位存在多个专用盘上。
缺点：冗余盘数量多，开销大，已淘汰。
RAID 3 (位交错奇偶校验)：
原理：按位分割。只有 1 个专用校验盘。
特点：任何一次读写都要所有盘同时转动。适合大文件连续读
写。
RAID 4 (块交错奇偶校验)：
原理：按块 (Block) 分割。只有 1 个专用校验盘。
缺点：每次写数据都要更新那个校验盘，导致校验盘成为性能
瓶颈（容易坏）。
RAID 5 (分布式奇偶校验) —— 主流：
原理：按块分割。校验信息分散存放在所有盘上（没有专用的
校验盘）。
优点：解决了 RAID 4 的瓶颈，读写性能好，空间利用率较高
（N-1/N）。
软件层面优化: 提前读\延迟写\虚拟盘
CHAPTER 5 文件管理
文件
文件：一组有标识（文件名）的信息集合
UNIX支持的文件类型
普通文件：ASCII， Binary
目录文件：文件目录组成的系统文件
特别文件
属性
基本属性：名称、扩展名、owner、group。
类型属性：魔数 (Magic Number, 用于识别文件类型)、
ASCII/二进制标识 。
管理属性：创建时间、最后存取时间、最后修改时间 。
控制属性：逻辑记录长、文件当前长/最大长、关键字位置 。
保护属性：读写执行权限、定时转储/多副本 (防系统崩溃)、
访问控制列表 (防非法操作)
UNIX 权限位 (10位) ：
格式：- rwx rwx rwx (第1位是文件类型，后9位分三组)
分组：Owner、Group、Others
文件目录
文件系统三层结构
文件管理层: 用户逻辑部分
目录管理层: 负责查找文件
磁盘主存映射管理: 逻辑地址->物理地址
文件控制块 FCB: 每个文件独有, 包含文件名等文件信息
文件目录: 依靠FCB实现文件的按名存取
UNIX下的目录项
当文件被打开时，外存 Inode 会被复制到内存。
inode新增字段：
i_flag：活动标识（占用、修改、上锁）。
i_count：引用计数（当前有多少进程打开了它）。
i_dev：设备号。
使得目录文件很小，检索极快
UNIX树形目录结构
特殊项
每个目录创建时自动生成两个项：
“.”：指向目录本身的 Inode。
“..”：指向父目录的 Inode。
根目录特征：“.” 和 “..” 指向同一个 Inode。
文件组织与数据存储
文件组织
流式文件：文件内数据为一串信息的集合，无结构
记录式文件：有结构的文件（高级语言 or DBMS提供）
物理结构
物理文件构造方法
计算法：将记录键映射到物理地址
指针法：设置专门指针指明记录物理地址
顺序文件：逻辑上连续的信息在存储介质上相连
缺点：需要预知文件长度，增删改困难
优点：顺序存取速度快
变种
紧凑顺序文件：记录连续存放，插入删除需移动大量记录。
扩展顺序文件：预留空白区域（备用），方便添加记录。
连接顺序文件：设置溢出区，新插入的记录通过指针链入溢出区。
连接文件：使用连接字（指针）连接各个记录
优点：便于增删改，不要求连续
缺点：指针&数据混合存放，破坏数据完整性。需要获取指针
才能访问下一物理块，仅适用于顺序存取
OS/2 改进：把指针块（FAT表）和数据块分离。指针块集中
存在磁盘首部，可以一次性读入内存（高速缓存），大大加快
检索速度。
直接文件：记录的关键字与地址（哈希）
索引文件：每个文件一个索引表，每个表目包含一个记录的键&存储地址
优点：便于增删改，不要求连续
缺点：增加索引表的空间开销&查找时间
UNIX 混合索引分配
结构：i-node 中包含 15 个 索引项（每项 4 字节）。
第 0-11 项：直接寻址 (直接指向物理块)。
第 12 项：一次间接 (指向一个存满物理块号的索引块)。
第 13 项：二次间接。
第 14 项：三次间接。
计算能力 (假设块大小 1KB，即1024字节)：
可以同时采用一级、二级、三级索引分配结构，以满足不同大
小文件的需要
文件系统其他功能
文件系统调用
文件系统提供给用户一系列系统调用
具体实现
建立文件
建inode：在设备上申请一个新的 i-node。
填属性：写入文件名、属性、权限。
分空间：调用辅存管理程序，分配第一个物理块。
登记：在目录中增加目录项，在活动文件表中登记信息。
打开文件
申空项：在活动文件表中申请空项，存放该文件的文件目录信息
查目录：根据文件名检索目录，找到外存 inode。
复制：将外存 Inode 复制到内存的活动表。
if 是共享文件，则共享用户 +1
文件定位，卷标处理
注意：Open 操作通常不进行数据读取，只是建立管理连接。
读写文件
按文件从活动文件表中找到文件目录
按存取控制说明检查访问合法性
逻辑记录-》物理块号
像设备管理发出IO请求，完成数据传输
关闭文件
将活动文件表中该文件的用户数-1；若为0，则撤销此表目。
若表目内容被改过，应先写回。
删除文件目录文件中删目录项，释放文件存储空间
UNIX文件系统调用
创建
删除
打开
关闭
读
写
文件卷的安装和使用
UNIX的文件系统分为基本文件系统（硬盘） & 可装卸的子文件系统
（文件卷，如软盘、U盘）
Windows和DOS OS下一个盘在物理安装后可直接使用
but UNIX和 Linux的每个文件卷需要安装后才能使用
文件卷安装
动态安装：将一个文件卷挂载到现有目录树的某个节点（挂载
点）上。
覆盖：挂载后，该节点原有的内容被暂时遮盖，变成了新文件
卷的根目录。
文件共享
静态共享（硬链接）：不同目录下的目录项（文件名不同），其 Inode
号 指向同一个物理 Inode 。是一种在磁盘存储层面的共享方式。
动态共享（并发访问）：是一种在内存运行层面的共享，指的
是“不同用户进程并发地访问同一个文件” 。
符号链接共享（软链接）：文件的内容为另一个文件的路径
名，访问时根据路径找目标文件。它有自己的 Inode，和原文
件的 Inode 不同。
辅存空间管理
连续分配：用户建立文件时，需给出文件大小
非连续分配：以块（扇区）为单位 or 以簇（若干扇区）为单位
内存映射文件
机制：由 OS 的虚存管理和文件系统共同实现。
缺页调度：当程序访问映射的虚拟地址时，若数据不在内存，
触发缺页中断，由存储管理子系统去磁盘加载页面。这把文
件 I/O 变成了内存访问。
虚拟文件系统 (VFS)
抽象层：VFS 定义了一套与具体文件系统无关的数据结构
作用：使得 Linux 可以同时挂载 ext2, ext3, fat32, nfs 等不同
文件系统，用户用同一套 open/read 接口操作它们。VFS 在
内核中进行转换。
CHAPTER 6 并发程序设计
BASIS
顺序程序执行
执行的顺序性：一个程序子啊处理器上的执行严格按序
环境的封闭性：运行程序独占全部资源
结果的确定性
过程的可再现性
并发程序设计
分类
并发多线程
并发多进程
特性
并发性
共享性：多进程可共享变量
交互型：多进程并发执行存在制约
制约关系
无关性：一组并发进程在不同变量集合上操作，互不影响。则
执行与时间无关（Bernstein条件） A不读B会写的，B不读A会写的，AB不写同一变量
防死锁（等待对方释放资源）
交互性：共享某些变量，有制约关系，需加以控制（进程间交互）
如果执行速度无法相互控制，会导致与时间相关的错误：结果
不唯一 or 永远等待
互斥：竞争资源
同步：协调执行
临界区
定义：临界区是指一个进程（或线程）中访问共享资源的那段
程序代码。
调度：
空闲让进：临界资源空闲让进程进入，不发生“互斥礼让”行为
忙则等待：同一时刻最多只有一个执行单元进入临界区。
有限等待：进入时间有限
让权等待：进程等待进入时应放弃CPU使用
算法（软件）
单标志法：无法多进程互斥。两进程必须交替进入，A进程结束，若B
进程进入改变标志，则A无法再进入。（违背空闲让进）
双标志先检查法：两进程各一个标志位表示是否正在访问临界
区。但两进程会一起进入。（违背忙则等待）
初始均为false
检查是指通过 while检查对方是否想进去
双标志后检查法：都无法进入，造成饥饿。（违背空闲让进和
有限等待）初始均为false
皮特森算法：一个flag数组（flag[i] = true，表示进程想要访问临界区），
和一个标志量turn（turn=i时，表示进程i有权进入临界区）。
初始flag全0，turn=-1
两进程同时运行，p0为i，自身置true，谦让turn=j，p1执行
第一行，p0先进入临界区
硬件实现临界区管理
软件难以实现多进程互斥控制
硬件指令方法
硬件设施
中断屏蔽
进程进入临界区前关中断，其他进程无法中断
but 影响效率，若用户忘记开中断会导致系统终止，且不适用
多CPU系统（其他处理器仍会执行临界区代码）
测试并建立指令
一条原子指令：合并了加锁与检查的操作
old = *lock
*lock = true
return old
如果lock为true，则返回true，表示已被占用，进程循环等待
如果为false，返回false，表示可进入，lock设置为true
利用TS（Test & Set）实现互斥锁，通过一个布尔锁变量和原
子操作（读取lock，并置true上锁，当false时，其他进程才可
进入），确保同一时间只有一个进程进入临界区，但存在忙等
和饥饿问题。
忙等待：如果锁一直被占用，不断循环检测 lock，浪费CPU时
间片
违背有限等待：进程可能抢不到锁，产生饥饿
对换指令
一条原子指令swap。它可以原子地交换一个寄存器的值和一
个内存地址的值
进程可以将一个“锁定”标记（如 1）放入寄存器，然后与内
存中的锁变量（初始为 0）进行交换。通过检查交换回寄存器
的值，进程可以知道自己是否成功获取了锁（如果换回来的
是 0，则成功）。
使用硬件指令方法实现互斥
优点
简单，易于检验正确性
适用任意进程数，适用多处理器系统
支持多临界区（为每个临界区设置一个布尔变量）
缺点
违背“让权等待”（占用while循环检查）
饥饿
互斥锁 mutex lock
通过acquire()和release()两个原子操作, 控制临界区, 每个互斥
锁有一个布尔变量available, 如果为false,则可以安次区, 否则
忙等.
优点: 适用多处理器, 等待锁期间无上下文切换(单纯while等待
available)
PV操作
进程互斥
信号量(Semaphore): 控制PV操作执行的变量, 表示物理资源
实体. 除赋初值, 仅可通过同步原语(PV操作)操作整数: 资源可复用次数
0: 无资源且无进程等待
负数: 等待队列中的进程数
PV操作原语
整型信号量
wait(S):
 while (S<=0);
 --S;
signal(S):
 ++S;
存在忙等
记录型信号量 W(s.list)调用block原语, 插入等待队列, 遵循"让权等待"
R(s.list)调用wakeup原语, 唤醒队列中第一个进程
P: wait()
V: signal()
机票问题
进程同步
同步: 对多个并发进程（或线程）的执行时序进行协调和制
约，确保它们按照预设的正确次序运行。(步调一致, 合谐工作)
同步亦称直接制约关系，是为某任务建立多个进程，进程因需
要协调运行次序而等待、传递信息所产生的制约关系。
若某行为提供资源, 则在该行为后V操作
若某行为需要该资源, 则在行为前先P操作
semaphore S=0;
P1():
 x;
 V(S);
P2():
 P(S);
 y;
进一步扩展: 课实现前驱关系
Si():
 P(a1);
 V(a2);
 V(a3);
说明a2, a3在a1后面
信号量仅仅解决信号传递
数据传送需要共享缓冲区
例子
生产者-消费者问题
1生产者1消费者1缓冲区
1生产者1消费者N缓冲区 + 队列指针
N生产者N消费者N缓冲区 + put & get 的互斥信号量
苹果橘子问题
读写问题写者优先 P操作-1, V操作+1
rdr 信号量：这是通往等候室的前门。一次只允许一个人通过
这扇门。写者可以锁上这扇门，从而阻止任何新的读者进入等
候室。
wrt 信号量：这是通往主阅览室的大门。第一个进入的读者会
锁上它，以阻止写者进入。最后一个离开的读者会解锁它，以
允许等待的写者进入。
写者通过首先锁住前门 (rdr) 来获得优先权，这确保了没有新
的读者可以排在当前读者之后。然后，写者等待主阅览室
(wrt) 清空。
哲学家问题
AND型信号量
思想: 一个进程所需资源, 全部分配 or 全部不分配. 防止都拿了
一部分资源, 等其他人释放资源, 造成死锁.
全部条件满足再执行
系统将原子性地完成以下操作：
首先，不会获得任何资源，以破坏死锁的‘请求与保持’条件。
随后，进程的状态由‘运行态’转为‘阻塞态’，并被挂载到它遇到的第一个不满足条件的信
号量的等待队列上。
与此同时，操作系统会在该进程的进程控制块（PCB）中，保存 Swait 这条指令的起始地址作
为其被唤醒后的重新执行点。
进程在此之后将放弃CPU，直到被其他进程唤醒。
signal唤醒所有相关进程, 让他们自己竞争
管程(Monitor)
BASIS
管程, 即资源管理程序, 是一种进程同步工具. 定义了共享资源
的数据结构及其上的一组操作.
管程: 封装同步操作(大量分散的PV操作), 对进程隐藏同步细节, 简化
调用. 如Java中的synchronized关键字
看管临界区，让访问者有序访问操作
封装操作，访问者只用在乎做什么即可，不用管怎么做和同步
问题
signal处理
执行 signal 的进程（A）立即放弃执行权，并被挂起。被唤醒
的进程（B）立即获得执行权并开始执行。
执行 signal 的进程（A）继续持有执行权，直到它自己主动离
开管程或执行 wait。被唤醒的进程（B）并不会立即执行，而
是从“条件变量的等待队列”被移动到“管程的入口等待队
列”，准备重新竞争。(现代编程语言常用)
生产者-消费者问题实例
class BoundedBufferProblem:
public static void main(String args[]) {
 //声明管程productBuffer实现对缓冲区的集中管理
 PorductBuffer pb = new PorductBuffer():
 newProducer(pb); //生产者和消费者使用同一个管程
 newConsumer(pb);
}
class Producer implements Runnable
ProductBuffer pb;
Producer(ProductBuffer_pb) {
 this.pb = _pb;
 new Thread(this,，"Producer").start();
}
public void runO）{
 int i=0;
 while(true)
 pb. put (i++) ;
}
class ProductBuffer {
 int n; // 缓冲区只有一个整数值
 boolean valueSet = false; // 标志位，表示缓冲区中是否有值
 synchronized int get() {
 if (!valueSet) wait();
 valueSet = false; // 标记缓冲区变空
 notify();
 return n;
 }
 synchronized void put(int n)
 if (valueSet) wait();
 this.n = n;
 valueSet = true; // 标记缓冲区变满
 notify();
 }
}
条件变量: 管程内的数据结构, 只能通过两原语(wait()和
signal())操作. 可确保进程互斥访问, 实现同步机制.
一个管程中可有多个条件变量(进程可能因为多个原因被阻塞),
若阻塞则挂起至对应条件x
与信号量的区别: 条件变量没有值, 仅实现排队功能; 信号量有
值, 表示剩余资源数
霍尔管程
进程通信
通过信号量实现的进程互斥&同步时低级通信方式, 传输信息
量小，效率较低
进程通信机制
分类
进程直接通信
原语:
进程间接通信
通过信箱进行中间转发
结构：信箱分为 信箱特征（容量、格式、指针）和
信箱体（存放信件的分区）
处理流程
发送：若信箱未满，存入信件并唤醒等待者；若满，发送者阻
塞（等待信箱） 。
接收：若信箱有信，取出一封并唤醒发送等待者；若空，接收
者阻塞 。
基于流的进程通信
多个进程共享一个消息缓冲区, 写入/读出字符流
远程过程调用 (RPC)
基于C/S计算模式, 服务器提供过程.服务供客户进程调用
异构处理：引入 外部数据表示 (XDR)，负责在不同硬件架构
（客户机 vs 服务器）之间转换数据格式
分类：共享内存、消息传递、管道通信、共享文件（P65）
例题
能保证进程互斥进入临界区，且不会出现“饥饿”现象
死锁
产生
独占资源被多个进程共享, 处理不当导致永远相互等待, 产生死锁
类别
进程推进顺序不当（请求和释放资源的顺序不当）
进程请求多种资源的顺序不同，形成循环等待链。
T1, P1申请并获得R1
T2, P2申请并获取R2
T3, P1在持有R1的同时, 申请R2, 但R2被P2占有, so P1必须进
入阻塞态
T4, P2在持有R2的同时, 申请R1, 但R1被P1占有, so P2必须进
入阻塞态
=>产生死锁
PV操作使用不当产生死锁对多个信号量（锁）的P操作顺序不一致。
资源分配不当引起死锁
由于系统资源分配策略不合理，或者可供分配的资源总量不能
满足所有进程并发执行的需求所导致的。
对临时性资源使用不加限制引起死锁
通信协议设计不当，进程间互相阻塞等待一个永远不会发生的
通信事件（如消息或信号）。
注: T表时刻, P表进程, R表资源
防止(破坏死锁发生的必要条件) 产生死锁的必要条件
互斥条件: 资源无法共享
不可剥夺条件改成可剥夺：实现复杂，适用状态易于保存和恢复的资源
占有和等待条件(请求并保持): 申请资源时不释放已有资源
A：静态分配策略: 进程执行前申请所有需要资源缺点: 提前分配了后续需要的资源, 降低了资源利用率
B：进程只获得初期所需资源，过程中逐步释放用完的资源
后，才能申请新的资源
循环等待条件(环路条件): 存在一个由两个或多个进程组成的等
待环路。其中，每个进程都在等待下一个进程所占有的资源。层次分配策略（顺序资源分配法）
申请资源的方向杂乱无序会导致产生环路, 所以要制定方向, 如
果要回头就需要释放所有资源, 形成有向无环图
给各资源编号，只能按编号递增申请资源
but 如果两进程的资源使用顺序相反, 其中
之一就不能正常获得资源
方式一：提前申请，无效占用提前申请R1和R2
方式二：释放已有，重新申请
申请R2, 使用R2执行部分任务
释放R2 (需要R1了)
申请R1
再申请刚才使用的R2
使用R1
避免死锁, 但失去效率
避免(分配时判断是否会导致不安全状态) 银行家算法
思想: 先预演, 再分配
银行家算法时一种动态策略, 在每一次
分配资源时进行安全性检查
避免死锁的实质: 系统在分配资源时, 保证系统不进入不安全状态
先决条件: 进程进入系统须生命最多需要多少资源
数据结构
Max(最大需求矩阵) Max[i][j] = k 表示进程 i 最多需要 k 个 j 类资源
Allocation(已分配矩阵) Allocation[i][j] = k 表示进程 i 当前已分配了 k 个 j 类资源
Need(需求矩阵) Need[i][j] = Max[i][j] - Allocation[i][j]
Available(可用资源向量) 记录了当前系统里，每一种资源还剩下多少个可供分配
算法工作流程
资源请求算法
1. 是否越界: 比较 Request_i 是否小于等于 Need[i]
2. 资源是否足够: 比较 Request_i 是否小于等于 Available
3. 尝试分配(预演): 假装分配, 临时更新
4. 执行安全性算法进行判断
安全性算法
1. 初始化: 创建向量 Work = Available, 创建向量 Finish 并初
始化 all 进程为false表未完成
2. 寻找可完成进程: 要求进程Pi : Finish[i] == false &
Need[i] <= Work
3. 执行完Pi并释放所有占用资源, 更新Work = Work +
Allocation[i], 标记 Finish[i] = true, 回到步骤2继续寻找.
if 找不到, 若所有进程均finish, 则找到哦一个安全序列, 系统处
于安全状态; 若有未完成的, 则系统处于不安全状态.
缺陷(缺乏实用价值):
- 很难在进程运行前知道所需最大资源量
- 进程间须无关
- 进程个数和分配的资源数须固定
检测&解除
通用操作系统, 采取"鸵鸟算法" 不预防, 不避免, 处理权交给用户
环路判断
情况一：每个资源类只有一个资源（环路是死锁的充要条件
情况二：资源类中有多个资源（环路是死锁的必要不充分条
件）
简化检测方法
P1持有2个R1资源, 请求1个R2资源
P2持有1个R1和R2, 请求1个R1和R2
P1的需求可满足, P2不行, 被阻塞.
so P1非阻塞非独立, 让其先执行, 删去其所有边
再看P2, 可满足了
执行P2, 删去其所有边
若能让 all 结点都成为孤立点, 则该图是可完全简化的
死锁定理: 不可完全简化 -> 系统为死锁状态(充分条件)
不可完全简化: R3存在资源冲突
死锁的解除策略
结束所有进程，并重启操作系统代价最高，丢失所有工作
撤销所有陷于死锁的进程代价高，数据可能不一致
逐个撤销陷于死锁的进程
代价相对较低
算法开销大，需要反复检测
剥夺陷于死锁进程的资源
不用杀死进程，保留工作
实现复杂，如何处理被剥夺者是个难题
进程回退至检查点
最安全，保证数据一致性
机制复杂，系统开销巨大
图例
Red Star
Pin
Idea